n=2, k=2

[1,2] [2,1] [1,2]

n=3, k=3.

[1,2,3] [3,1,2] [2,3,1] [1,2,3]

n=k

[a,b,c,....].

CyclicRotation

Ideas

A = [a1, a2, a3, ..., an]

Special case, if n=k, → do nothing. return.

---

if n≠k., do something.

If. a1=a2=a3=...=an → do nothing. return

How to rotate.

last element pulled out.

Starting from first element, shift to right by 1., until the end of list..

Pseudocode for Cyclic Rotation.

```
size = len(given_list).
IF size = len
    RETURN given_list

IF every element is the same,
    RETURN given_list
```

last_element = A[size-1]

~~FOR e in range(1, size-1)~~

~~temp = A[e]~~

~~A[~~

FOR e in range(size-1, 0). # do it from the end.

A[e] = A[e-1]. → e-1 ≥ 1

A[0] = last_element

RETURN A

A[0] → A[n].

A[1] → A[0]

A[2] → A[1].

⋮

[0, 0, 0, 0]

Odd Occurrences In Array.

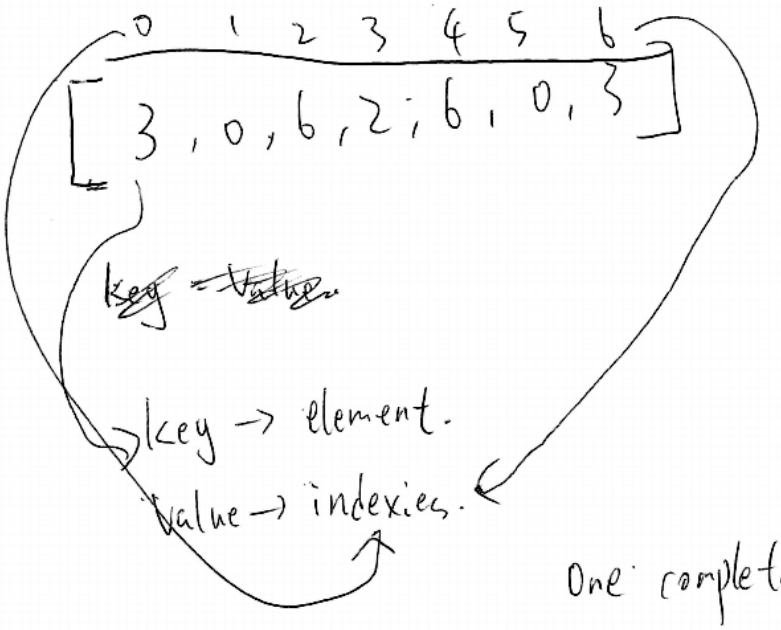
Ideas: $A = [a_1, a_2, \ldots\ldots a_n]$ $a_i \in \mathbb{Z}$

$|A| = 2n+1$

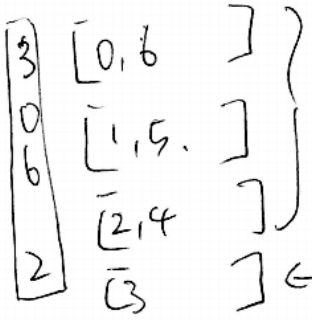
Find the index of unpaired element.

Efficient? O(n)?

Tools? Dictionary?



One complete pass.



even $\%_2 = 0$.

size of value array. $\%_2 = 1$.

↓

return the key.

← odd $\%_2 = 1$

One iteration + one dictionary

~~FOR i in range(A).~~

Pseudo code for odd occurrens in array.

SET Dictionary, key as A's element, value as list of element idx.

~~For i in range(A).~~

~~dictionary key~~

~~IF A[i] in dictionary~~

push

~~For i~~

For i in range (A)

IF A[i] not in dictionary.

push A[i] to dictionary.

IF A[i] in dictionary.

push i to dictionary using A[i] as key.

FOR v in dictionary.

If V's ~~key~~ value odd size.

return. V

Frog Jump. at x., want to go to y or $\geq y$.

$$x + tD \geq y.$$

↑

find t.

special case. $x = y$.

then return 0.

$x < y$. need movement.

$$t \geq \left\lceil \frac{y-x}{D} \right\rceil = \text{solution}.$$

Pseudo Code for frog jump.

```
If x=y
    return 0

IF y>x
    Return ⌈(y-x)/D⌉.
```

Perm Missing Element.

Ideas: ~~A = [1, 2, 3, ... n]~~ -

$A = \underbrace{1, 2, 3, \ldots n}_{\text{sorted / unsorted.}} - x$.

If A is not missing any element then Sum of elements of A is.

$$\frac{N(N+1)}{2}.$$

Thus. the missing element $= \frac{(N)(N+1)}{2}$ - Sum of A.

Pseudo code for Perm Missing Elem.

```
Sum = A.sum()
Return (N)(N+1)/2 - sum.
```

Tape Equilibrium.

$A = [a_1, a_2, \ldots a_n]$.

Minimal diff ↓

weight | weight. ?

which groups of numbers ~~A=~~ $|B-C| = 0 \rightarrow 0$ close to 0.

Fist. know the available numbers.

from $0 \rightarrow N$

from $N \rightarrow 0$.



possible combo.

A[1]	A[2 ... N].
A[1-2]	A[3 ... N].
A[1-N-2]	A[N-1+N]
A[1-N-1]	A[N]

} can be done in one pass.

Exhaustive? ~~Yes.~~ cut in middle, imbalance. ~~left/righ~~

so on so forth.

Avoid Exhaustion.

$B > C$

$B = C$

$B < C$.

Pseado Code for Tape Equilibrium.

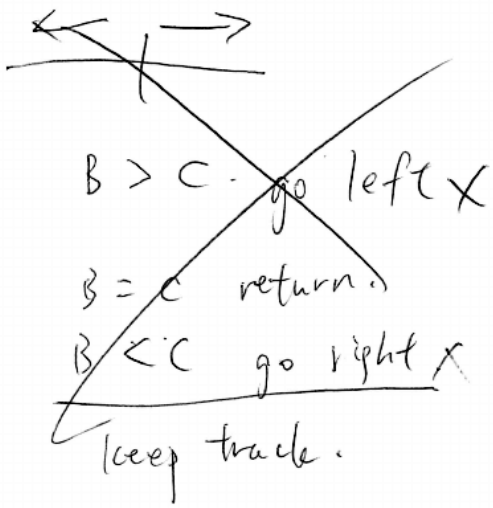
~~P = len(A) // 2~~

~~sum_B = 0~~

~~sum_C = 0~~

~~For I in range(len(A) - 1)~~

~~Su~~



SUM_B = sum(A[0:P-1])

SUM_C = sum(A[P:len(A)-1]).

IF sum_B > SUM C :

When to stop?

B = C,

~~previous diff is better than~~

~~this time~~ - -

negative nums fails.

Consider negative nums.

mini_diff = ~~0~~

size = len(A) - 1

Sum_B = 0

Sum_C = 0.

While

One pass.

min_diff = ∞

left_sum = 0

right_sum = sum(A).

size = len(A).

FOR P in [1, size - 1].

value = A[P-1]

left_sum = left_sum + value.

right_sum = right_sum - value.

min_diff = MIN(min_diff , ABS(left_sum - right sum)

IF min_diff = 0 , RETURN 0

→ RETURN min_diff .